# 1.3 Vectors

Vladimir Buskin

English Language and Linguistics, Catholic University of Eichstätt-Ingolstadt

ORCID: 0009-0005-5824-1012

Last updated: 29 September 2026, 21:45

We introduce vectors as our first data structure in R and explore some common applications and manipulations.

## Table of contents

Preparation . . . . . 1
Recommended reading . . . . . 2
Word frequencies I . . . . . 2
Storing data in R . . . . . 2
Creating the barplot . . . . . 3
Essential R concepts . . . . . 5

**Exercises** **6**
Tier 1 . . . . . 6
Tier 2 . . . . . 7
Tier 3 . . . . . 8

**References** **10**

### Preparation

Script

You can find the full R script associated with this unit here.

## Recommended reading

Baayen (2008): Chapter 1

Winter (2020): Chapter 1.1–1.9

Suggested video tutorial:

How to Create and Name Vectors in R (DataCamp; 5min)

## Word frequencies I

In usage-based linguistics, it is very common to work with word frequency data of the kind shown in Table 1.

Table 1

| Lemma | Frequency |
|---|---|
| start | 418 |
| enjoy | 139 |
| begin | 337 |
| help | 281 |

While this table is relatively small and easy to interpret, it is usually a good idea to supply readers with a simple visual representation as well. The more complex the data becomes, the greater the value of clear visualisations will be. When dealing with counts of distinct categories as it is the case here, we can draw on the primary workhorse of categorical data analysis – the **barplot**.

### Storing data in R

To create a two-dimensional barplot, we will first need to generate two objects in R: one for the individual lemmas ($x$-axis) and one for the frequency counts ($y$-axis).

First, let's combine lemmas *start, enjoy, begin* and *help* into a single virtual object using R's `c()` function, which you can read as 'concatenate' or 'combine'. We will call this new object `lemma`. Enter the following line into a new R script and click on **Run** (or simply press Ctrl+Enter/Cmd+Enter).

```
lemma <- c("start", "enjoy", "begin", "help")
```

To make sure everything is working as intended, we can apply the `print()` function to the `lemma` object, in order to display all the elements contained within to the console:

```
print(lemma)
```

```
[1] "start" "enjoy" "begin" "help"
```

Naturally, we can also combine numeric data with `c()`:

```
frequency <- c(418, 139, 337, 281)
```

Once again, the `print()` function allows us to inspect the contents of `frequency`:

```
print(frequency)
```

```
[1] 418 139 337 281
```

> When do I use quotation marks?
>
> Letters and numbers represent two distinct data types in R. Anything that should be understood as a simple sequence of letters must be enclosed by quotation marks `"..."`. A linguistic item such as *start* will be evaluated as a **string** if it's encoded as `"start"`. **Numbers** (or **integers**), by contrast, appear without quotation marks.

Our linguistic data is now stored in two **variables** `lemma` and `frequency`, which you can conceptualise as virtual container-like objects. These 'containers' are now showing in the **Environment** tab in the top right corner of your RStudio interface (cf. Figure 1).

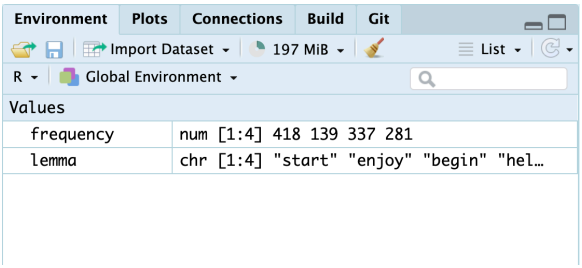



Figure 1

### Creating the barplot

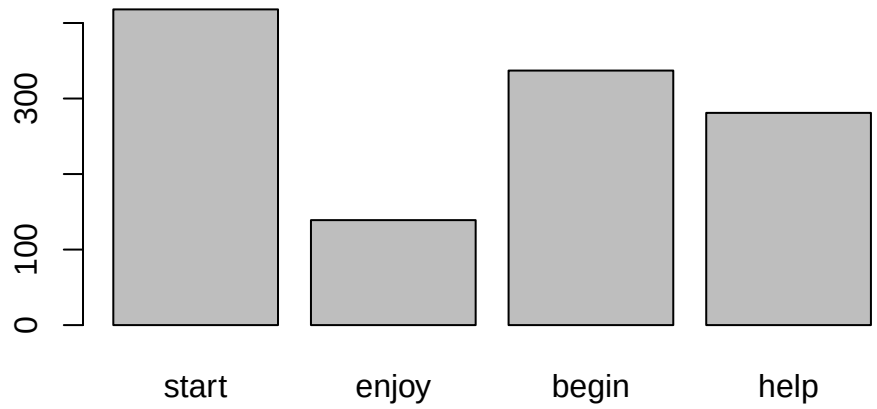
A function in R can take one or more **arguments** to which it will be applied. R's most basic barplot function (which is, unsurprisingly, called `barplot()`) needs at the very least …

- a `height` argument, i.e., our y-axis values and

- a `names.arg` argument, i.e., our x-axis labels.

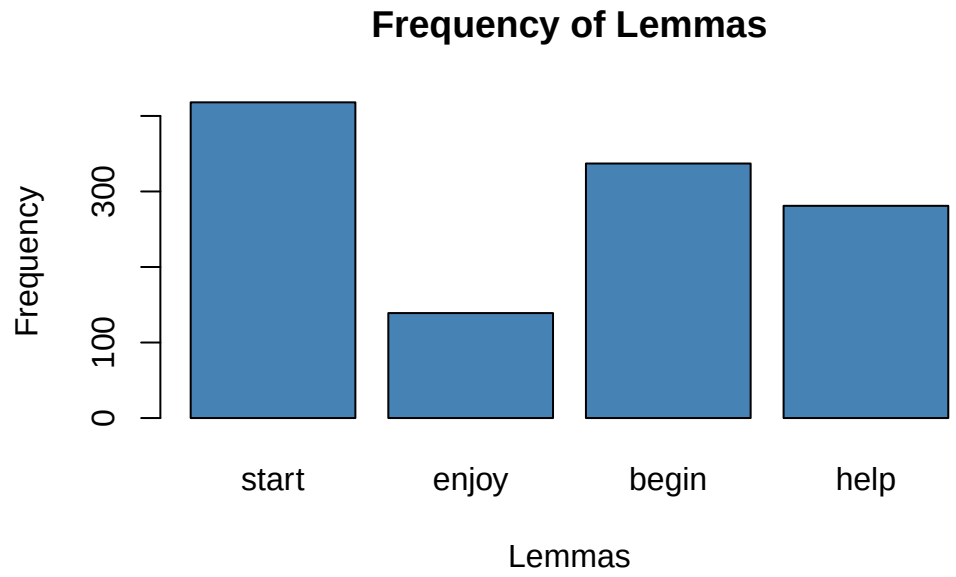
The function arguments must be enclosed by parentheses and separated by commas:

```
barplot(frequency, names.arg = lemma)
```




This plotting function supports several additional arguments which can be used to customise the plot further. Typing `?barplot` into the console opens a (mildly overwhelming) help tab that offers a detailed breakdown of all customisation options. After some tinkering, our barplot looks more presentable:

```
barplot(frequency, names.arg = lemma,
        main = "Frequency of Lemmas", # title
        xlab = "Lemmas",  # label for x-axis
        ylab = "Frequency", # label for y-axis
        col = "steelblue") # color
```



> What does '#' mean? On comments in R
>
> In R, everything followed by the hashtag `#` will be interpreted as a comment and won't be evaluated by the R interpreter. While comments don't affect the output of our code in the slightest, they are **crucial** to any kind of programming project.
> Adding prose annotations will make your code not only easier to understand for others but also for your future self. Poor documentation is a common, yet unnecessary source of frustration for all parties involved …

In RStudio, you now have the option to save the plot to your computer. Once the figure has appeared in your "Plots" panel, you can click on "Export" in the menu bar below and proceed to choose the desired output format and file directory.

### Essential R concepts

The example above demonstrates one of the most important data structures in R: **vectors**. They form the cornerstone of various more complex objects such as data frames, and are essential to handling large data sets (e.g., corpora). And yet, vectors are very simple in that they are merely one-dimensional sequences of characters or numbers — no more, no less.

```
print(lemma)
```

```
[1] "start" "enjoy" "begin" "help"
```

```
print(frequency)
```

```
[1] 418 139 337 281
```

The individual elements in these two vectors are not randomly jumbling around in virtual space, but are in fact following a clear order. Each element comes with an "ID" (or **index**), by which it can be accessed. For example, if we want to print the first lemma in our `lemma` variable, we append square brackets `[ ]` to it. This will allow us to **subset** it.

```
lemma[1]
```

```
[1] "start"
```

Similarly, we can subset `frequency` according to, for example, its third element:

```
frequency[3]
```

```
[1] 337
```

It is also possible to obtain entire ranges of elements, such as everything from the second to the fourth element:

```
frequency[2:4]
```

```
[1] 139 337 281
```

To check the number of elements in a vector, we use `length()`:

```
length(lemma)
```

```
[1] 4
```

```
length(frequency)
```

```
[1] 4
```

## Exercises

> Solutions
>
> You can find the solutions to the exercises here.

### Tier 1

**Exercise 1.** Create a vector that lists the third person personal pronouns of English (subject and object forms). Store them in a variable `pp3`.

**Exercise 2.** Now print …

- … the fourth element in `pp3`.
- … elements 3 through 5.
- … all elements.
- … elements 1, 3 **and** 5.

## Tier 2

**Exercise 3.** When working with large datasets, we often do not know whether an element is in the vector to begin with, let alone its position. For instance, if we wanted to check whether *they* is in `pp3` or not, we could use the handy notation below, which returns a `TRUE` or `FALSE` value:

```
"they" %in% pp3
```

Ascertain whether the following items are in `pp3`:

- *him*
- *you*
- *it* and *them*
- *we*, *us* and *me*

**Exercise 4.** Once we are sure that an element is in the vector of interest, another common problem that arises is finding its location. In this case, we can use `which()` to return the index of an element.

```
which(pp3 == "they")
```

You can read the code above as "Which element in `pp3` is *they*?". Note that the index number depends on the order of elements you've chosen when creating `pp3`. Find the positions of *it* and *them* in `pp3`!

**Exercise 5.** Consider the vector `numbers`.

```
numbers <- c(500:1000)
```

- Explain the difference in output for the following two code lines:

```
which(numbers > 600)
```

```
numbers[which(numbers > 600)]
```

- Examine the output of the code chunks below, and try to establish the meaning of the operators `!=`, `&`, and `|`.

```
numbers[numbers != 500]
```

```r
numbers[numbers > 500 & numbers < 550]
```

```r
numbers[numbers < 510 | numbers > 990]
```

### Tier 3

**Exercise 6.** Consider our frequency data again. When analysing linguistic patterns, we often need to transform our data. Starting with:

```r
lemma <- c("start", "enjoy", "begin", "help")
frequency <- c(418, 139, 337, 281)
```

- Create a new vector `relative_freq` that shows each frequency as a percentage of the total.
- Create a vector `log_freq` containing the natural logarithm of each frequency value.
- Suppose these frequencies come from a corpus of 10,000 words. Create a vector `freq_per_thousand` that shows how many times each verb appears per 1,000 words.

**Exercise 7.** The function `strsplit()` can be used to split up a string into substrings.

```r
words <- c("read", "write")

split_words <- strsplit(words, split = NULL)

print(split_words)
```

```
[[1]]
[1] "r" "e" "a" "d"

[[2]]
[1] "w" "r" "i" "t" "e"
```

Here the output is a **list** with two objects enclosed by double square brackets (`[[1]]` and `[[2]]`). These can be used to subset `split_words`:

```r
# Get first list object
split_words[[1]]
```

```
[1] "r" "e" "a" "d"
```

```r
# Get second list object
split_words[[2]]
```

```
[1] "w" "r" "i" "t" "e"
```

Write an R script that identifies the number of (orthographic) vowels and consonants for the words *consequence* and *parsimonious*.

**Exercise 8.** R allows users to write their own functions in order to automate certain tasks. Say, we need a function that automatically computes percentages for a numeric vector. If we break down the individual steps, it has to

1. take a numeric vector as input,
2. compute the sum of its values,
3. divide the vector values by the sum to get relative frequencies.
4. multiply the relative frequencies by 100, and
5. return the final vector with the percentages.

```r
# 1. Define function with input
get_pct <- function(vector) {

  # 2. Compute sum
  sum_values <- sum(vector)

  # 3. Divide vector values by the sum
  relative_freqs <- vector/sum_values

  # 4. Multiply relative frequencies by 100
  percentages <- relative_freqs * 100

  # 5. Return the percentages
  return(percentages)
}
```

Now we can apply it to a numeric vector of our choice:

```r
get_pct(frequency)
```

```
[1] 35.57447 11.82979 28.68085 23.91489
```

Write a `count_vowels()` function and a `count_consonants()` function that immediately computes the number of vowels or consonants, respectively, for any word supplied.

## References

Baayen, R. Harald. 2008. *Analyzing Linguistic Data: A Practical Introduction to Statistics Using r*. Camrbidge: Cambridge University Press.

Winter, Bodo. 2020. *Statistics for Linguists: An Introduction Using r*. New York; London: Routledge.